main.c

Share Run Output Clear

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void append_node(int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct
        Node));
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
```

```
10 -> 20 -> 30 -> NULL
Found 20 at position 2
Not found


=== Code Execution Successful ===
```

```
    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

void search_node(int data) {
    struct Node *temp = head;
    int pos = 1;

    while (temp != NULL) {
        if (temp->data == data) {
            printf("Found %d at position %d\n", data, pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
```

```
    }
    printf("Not found\n");
}

void display_list() {
    struct Node *temp = head;
    if (temp == NULL) {
        printf("List empty\n");
        return;
    }

    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    append_node(10);
    append_node(20);
    append_node(30);

    display_list();
    search_node(20);
    search_node(40);

    return 0;
}
```

Page No.
Date

Singly Linked List 2

Algorithm

1. start the program and create a Node structure with data and next
2. Initialize head = NULL
3. To append a node : create a new node move to the last node and link the new node
4. To search : start the from head, compare each node data until found or list ends.
5. To display : Traverse from head and print each node until NULL.
6. Exit

append_node (data)

- create a new node
- The list is empty, new node become head.
- otherwise, it moves to last node and links new nodes at the end.

Search_node (data)

- starts from head
- compares each node's data given value.
- If found, prints position
- If not prints "not found"

display_list()

- start from head . Print each node in order until NULL.